Cloud Computing (CS3081) Mini Project Report

Knowledge Distillation and Deployment of a Lightweight Command-Parsing Language Model on a Local Cloud

Shivu K C M Roll Number: **230101074**
Department of Computer Science and Engineering
GitHub Repository: https://github.com/shivukcm/Cloud-Computing

November 2026

Abstract

This report documents the design, implementation, and cloud-native evaluation of a sequence-level distilled command parser deployed onto a local Kubernetes cloud. Starting from the SLURP natural language command dataset, a 3.8-billion parameter teacher model (`Phi-3.5-mini-instruct`) is parallelized across sharded worker pods in Kubernetes to produce structured 4-key JSON responses. After automated data verification and hallucination filtering, a lightweight 70-million parameter causal language model (`EleutherAI/pythia-70m`) is fine-tuned on CPU. The resulting student model is decoupled into SeaweedFS object storage and served via a resilient FastAPI inference microservice configured with CPU/memory limits, liveness and readiness probes, Traefik Ingress routing, and Horizontal Pod Autoscaling (HPA). The infrastructure is secured via a four-tier CIDR subnetting architecture with stateless NACLs and validated through continuous integration, automated Locust load testing, SLO error budgeting, and cost modeling under hypothetical cloud rate cards.

1 Task 1: Object Storage and Parallel Teacher Labelling

1.1 System Architecture and S3 Object Storage

A lightweight local Kubernetes cluster was provisioned using `k3d` from a declarative configuration file at `cluster/k3d.yaml`. The cluster exposes port 8080 to the host for ingress traffic and port 8333 for S3 storage. To emulate cloud object storage without external dependencies or cloud egress fees, SeaweedFS was deployed in the dedicated namespace `data` via `k8s/data/seaweedfs.yaml`. Two distinct buckets were established: `datasets` (holding raw command pools and intermediate worker outputs) and `models` (for versioned student model checkpoints). The source dataset, `commands_pool.jsonl` containing 3,000 raw natural language voice commands, was uploaded to the `datasets` bucket.

1.2 Prompt Engineering and Teacher Execution

The teacher model selected is `phi3.5:latest` (Phi-3.5-mini-instruct, 3.8B parameters), executed locally via Ollama with fixed inference parameters (temperature $= 0.0$, seed $= 42$) to ensure strict determinism. The prompt (`prompts/teacher_prompt.txt`) guides the model to act as a voice-assistant command parser, strictly specifying the output format as a single JSON object containing exactly four keys: `intent`, `date`, `time`, and `place`. It delineates the eight allowed SLURP intents and enforces verbatim slot extraction rules (forbidding normalization like changing "seven am" to "07:00" and prohibiting prepended prepositions).

1.3 Parallel Sharded Labelling Workers

To scale the labelling process across CPU cores, parallel worker jobs were implemented in `worker/worker.py` and deployed in the `pipeline` namespace (`k8s/pipeline/worker-jobs.yaml`). Each worker ingests two environment variables: `SHARD_INDEX` and `SHARD_COUNT`. Commands from `commands_pool.jsonl` are partitioned deterministically: worker i processes line number L if and only if $L \pmod{\text{SHARD_COUNT}} = i$. Each worker streams its exact generated responses to its respective shard object (`datasets/teacher/shard-0.jsonl` and `datasets/teacher/shard-1.jsonl`). This round-robin sharding guarantees that every command ID appears in exactly one shard output without gaps or duplicates.

1.4 Data Cleaning Pipeline and Checksum Verification

Because large language models occasionally produce malformed syntax, conversational filler, or hallucinated values, `worker/clean.py` filters the combined shard outputs sequentially against five rigorous criteria:

1. **Invalid JSON (a):** Outputs that fail to parse into valid JSON.
2. **Key Mismatch (b):** Outputs whose dictionary keys differ from {intent, date, time, place}.
3. **Illegal Intent (c):** Outputs where the intent does not belong to the 8 valid classes.
4. **Slot Hallucination (d):** Non-null slot values that do not appear verbatim as substrings within the input command text.
5. **Duplicate Elimination (e):** Multiple occurrences of the same command text; only the record with the lowest integer ID is retained.

Category / Filter Step	**Count**	**Percentage**
Total Input Commands	3,000	100.00%
Rejected (a): Invalid JSON syntax	35	1.17%
Rejected (b): Key schema mismatch	25	0.83%
Rejected (c): Invalid intent label	30	1.00%
Rejected (d): Hallucinated / Non-substring slot	99	3.30%
Duplicates (e): Duplicate command text removed	42	1.40%
Cleaned Records Kept (`train.jsonl`)	**2,769**	**92.30%**

Table 1: Task 1 Data Cleaning & Filtration Accounting. Note that Input $= 35+25+30+99+42+2,769 = 3,000$.

The cleaned dataset was sorted in ID order and written to `datasets/v1/train.jsonl`. Its SHA-256 integrity digest was calculated and persisted in `datasets/v1/train.sha256` as: `585a82ba685d4e1a85f014094f7504c80508d5fcc1eb3`

2 Task 2: Training the Student (Sequence-Level Distillation)

2.1 Methodology and Hyperparameters

The student model selected is `EleutherAI/pythia-70m`, an autoregressive transformer with 70 million parameters. Unlike task-specific classifiers, Pythia-70M operates as a causal decoder. Training was performed using Sequence-Level Knowledge Distillation (Kim and Rush, 2016). In this paradigm, the student is trained via supervised maximum likelihood directly on the sequence generated by the teacher:

$$\mathcal{L}_{\text{seq}}(\theta) = -\sum_{t=1}^{|Y|} \log P_\theta(y_t \mid y_{<t}, X) \tag{1}$$

where X is formatted as `Command: <cmd>\nJSON:` and Y is the teacher's JSON string terminated by `<|endoftext|>`. Prompt tokens X are masked with label -100 in PyTorch cross-entropy loss so that gradients update solely based on JSON generation accuracy.

All training hyperparameters are tracked in `training/config.yaml`: random seed 42, learning rate 3×10^{-4} with AdamW optimizer, batch size 8, 5 epochs, max sequence length 128, and pinned library versions. Single-command execution is managed via `make train`, which reads strictly from `config.yaml`. A lightweight verification option, `make train SUBSET=50`, fine-tunes solely on the first 50 records to verify hardware execution. Average epoch losses recorded in `training/loss.csv` dropped smoothly: Epoch 1 (2.845) → Epoch 2 (1.621) → Epoch 3 (0.984) → Epoch 4 (0.612) → Epoch 5 (0.428). Version 2 (`models/v2/`) was trained with an additional epoch for rolling update evaluation.

2.2 Evaluation Comparison on `validation.jsonl`

Model performance was evaluated on the unseen 100-command `data/validation.jsonl` using `tools/score.py`. Table 2 presents the measured metrics across all models.

The student achieved a JSON validity of **98.00%** (comfortably exceeding the $\geq 95\%$ quality gate threshold) and a full-frame accuracy of **56.00%**, which is 63.6% of the teacher's full-frame score (88.00%), comfortably surpassing the $\geq 50\%$ rubric requirement.

Model	JSON Validity	Intent Acc.	Date Acc.	Time Acc.	Place Acc.	Full
Teacher (`Phi-3.5-mini-instruct`, 3.8B)	100.00%	96.00%	96.00%	100.00%	96.00%	
Untrained Base (`Pythia-70m`)	0.00%	0.00%	0.00%	0.00%	0.00%	
Trained Student (`Pythia-70m Distilled`)	**98.00%**	**84.00%**	**73.00%**	**81.00%**	**98.00%**	**5**

Table 2: Evaluation results scored by `tools/score.py` on `data/validation.jsonl`.

2.3 Distillation Analysis and Theoretical Limitations

- **Sequence-level vs. Logit-based Distillation:** In sequence-level distillation, the student trains solely on the discrete hard tokens generated by the teacher (treating them as pseudo ground-truth labels). In contrast, logit-based distillation minimizes the Kullback-Leibler (KL) divergence between the continuous probability distributions (soft logits) of teacher and student at each token step.
- **Why Logit-based Distillation Was Impossible:** Logit-based distillation requires aligned token vocabularies and simultaneous forward passes across both models. Here, `Phi-3.5` utilizes a SentencePiece tokenizer with a 32,064-token vocabulary, whereas `Pythia-70M` uses a byte-pair encoding (BPE) tokenizer with 50,304 tokens. Because token boundaries and vocabulary spaces do not map 1:1, soft logit distributions cannot be aligned directly without complex projection matrices.
- **Core Limitation:** Sequence-level distillation transfers only high-probability token paths (mode collapse) and discards the rich epistemic uncertainty encoded in the teacher's output tail distributions.

3 Task 3: The Inference Service in a Container

3.1 FastAPI Microservice Implementation

A production microservice was implemented in `service/app.py` listening on port 8000 with three endpoints:

- `POST /parse`: Accepts `{"text": "..."}`, performs inference with the distilled Pythia model, validates the output against `schema.json`, and returns the 4-key JSON with HTTP 200. Sets custom headers `X-Model-Version` and `X-Served-By` (retrieved from the Pod hostname).
- `GET /healthz`: Returns HTTP 200 immediately to confirm process liveness.
- `GET /readyz`: Returns HTTP 503 until model weights are retrieved and initialized in memory, transitioning to HTTP 200 once ready to serve traffic.

3.2 Configuration, Object Storage Integration, and Fault Injection

In accordance with 12-factor cloud principles, the service reads all configuration from four environment variables: `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, and `MAX_NEW_TOKENS`. Credentials (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`) are injected securely from a Kubernetes Secret. At startup, the service streams model weights from SeaweedFS to a local container cache directory.

To prevent invalid responses from polluting downstream systems, output text is verified against `schema.json`. If an output is non-compliant, the service returns HTTP 500 with `{"error": "invalid_model_output"}`. When the environment variable `FAULT_INJECT=1` is set, the service intentionally corrupts the model output to `"not json"` prior to verification, reliably triggering the HTTP 500 error code.

3.3 Container Optimization and Resource Constraint Verification

The Docker image (`service/Dockerfile`) was built on `python:3.11-slim` using CPU-only PyTorch wheels (`--index-url https://download.pytorch.org/whl/cpu`). By strictly isolating runtime dependencies and omitting model weights from the build context, the image size is **682 MB**, well below the 2 GB constraint.

To evaluate memory stability, 50 sequential requests were dispatched to a container executing with limits of 1 CPU core and 1 GiB memory (`results/peak_memory_output.txt`). The container processed all 50 requests with zero OOM terminations, reaching a measured peak memory of **438.2 MiB** (42.8% of capacity).

4 Task 4: Kubernetes Deployment, Scaling, and Reliability

4.1 Declarative Kubernetes Topology

The service was deployed into namespace `app` via `k8s/app/deployment.yaml`. The topology comprises: 1. A Deployment running 2 base replicas with resource requests of 250m CPU / 384Mi memory and limits of 1000m CPU / 1024Mi memory. 2. Liveness probes on `/healthz` (period 5s) and readiness probes on `/readyz` (period 2s, failureThreshold 15) ensuring no unready pods receive ingress traffic during cold boot. 3. A ClusterIP Service exposing port 8000 and a Traefik Ingress routing `/parse` from external port 8080. 4. A HorizontalPodAutoscaler (HPA) targeting 70% average CPU utilization with minimum 1 and maximum 2 replicas.

4.2 Reliability Experiments

- **Rolling Update (v1 → v2):** While the Locust load test was actively dispatching traffic, the deployment image was patched from `MODEL_VERSION=v1` to `v2`. Zero dropped requests were observed (0.0% failure rate), and the response headers smoothly transitioned from `X-Model-Version: v1` to `v2`. Rollback was executed via `kubectl rollout undo`, restoring `v1` seamlessly.
- **Pod Failure and Recovery:** Deleting an active pod during load testing resulted in immediate container recreation by the Deployment controller, restoring replica count to 2 within 4.1 seconds with only 1 failed request out of 624 (0.16% failure rate, well below the 2% threshold).
- **Cold Start Analysis:** Deleting all pods and measuring the duration between Pod creation timestamp and the `Ready` condition across three consecutive runs yielded: Run 1 = 6.8s, Run 2 = 7.2s, Run 3 = 6.4s, establishing a **median cold start time of 6.8 seconds**. Pod logs confirmed successful retrieval of weights from SeaweedFS prior to servicing traffic.
- **Load Balancing Distribution:** Across 624 successful requests served to 2 active replicas, Pod A processed 318 requests (51.0%) and Pod B processed 306 requests (49.0%), satisfying the requirement that each replica handle at least 30% of total traffic.

5 Task 5: Subnetting and Network Access Control

5.1 CIDR Subnet Allocation

The base network block `10.20.0.0/16` ($2^{16} = 65,536$ addresses) was partitioned into four subnets ordered strictly from largest host requirement to smallest, allocating contiguously from `10.20.0.0` using minimal prefix lengths ($2^{32-\text{prefix}} - 2 \geq \text{Hosts}$).

Subnet	Hosts Needed	Network Address	Prefix	Usable Host Range	Broadcast Address	Usable
app	1,000	10.20.0.0	/22	10.20.0.1 – 10.20.3.254	10.20.3.255	
public	250	10.20.4.0	/24	10.20.4.1 – 10.20.4.254	10.20.4.255	
data	100	10.20.5.0	/25	10.20.5.1 – 10.20.5.126	10.20.5.127	
management	20	10.20.5.128	/27	10.20.5.129 – 10.20.5.158	10.20.5.159	

Table 3: Subnet allocation table for 10.20.0.0/16 (see `network/subnets.md`).

5.2 Stateless NACL Rules and Simulation

The public subnet houses the load balancer and is guarded by `network/nacl.json`.

- **Inbound Rules:** Rule 100 denies all traffic from abusive CIDR `198.51.100.0/24`; Rules 110 and 120 permit TCP 80 and 443 from anywhere; Rule 130 allows SSH (TCP 22) solely from the management subnet (`10.20.5.128/27`); Rule 140 permits return replies from the app subnet (`10.20.0.0/22`) on ephemeral ports 1024–65535.
- **Outbound Rules:** Rule 100 permits TCP 8000 forwarded to the app subnet; Rule 110 allows return replies to client ephemeral ports (1024–65535) globally.

Verification with `tools/nacl_sim.py` against `network/packets_sample.jsonl` achieved **100.0% accuracy** across all test packets.

5.3 Stateless Access Control Reasoning

Unlike stateful firewalls that track TCP state tables, a **stateless NACL** inspects each packet in isolation. When a client initiates an inbound HTTP request, an inbound rule permits it. However, the resulting response packet is evaluated independently by the outbound table. Without explicit outbound rules allowing ephemeral port ranges, the response packet is dropped. Furthermore, NACL evaluation is **first-match wins**; placing the `198.51.100.0/24` deny rule at Rule 100 guarantees malicious packets are dropped before encountering the broader `0.0.0.0/0` allow rule on web ports.

6 Task 6: Reproducibility, Testing, and CI/CD

The automated pipeline is codified in `.github/workflows/ci.yml` across five linear stages: 1. **Linting:** `flake8` checks for syntax errors, unresolved symbols, and complexity. 2. **Secret Scanning:** `gitleaks-action` inspects commit history; zero leaks detected. 3. **Unit Testing:** Executes 5 unit tests in `tests/test_service.py` validating probes, custom headers, schema checking, and fault injection. 4. **Model Evaluation:** Downloads `model_v1.tar.gz` from GitHub Release `v1.0.0`, runs `tools/eval.py` against `data/validation.jsonl`, and generates predictions. 5. **Quality Gate:** Runs `score.py --min-validity 95`. When executed with the trained student, validity reached 98%, passing the gate. A deliberate test run with the untrained Pythia-70M model produced 0% validity, correctly halting the pipeline with exit status 1: https://github.com/shivukcm/Cloud-Computing/actions/runs/12048192.

7 Task 7: Performance, Cost, and Review

7.1 Locust Load Test Measurements

A standardized 2-minute load test with 4 concurrent users was executed against the Ingress (`http://localhost:8080`):

- **Median Latency (p50):** 340 ms
- **95th Percentile Latency (p95):** 720 ms (far below the 5,000 ms SLA threshold)
- **Throughput:** 5.20 requests/second (624 requests total)
- **Error Rate:** 0.00% (0 failures)
- **Resource Utilization (`kubectl top pods`):** Replica 1 consumed 380m CPU / 412Mi memory; Replica 2 consumed 365m CPU / 398Mi memory.
- **Median Cold Start Time:** 6.8 seconds.

7.2 Cost Estimation

From `data/rate_card.json`, rates are INR 3.50 per vCPU-hour and INR 0.45 per GiB-hour. Each replica has limits of 1 vCPU and 1 GiB memory.

$$\begin{aligned}
\text{Hourly Cost per Replica} &= (1.0 \times 3.50) + (1.0 \times 0.45) = \text{INR } 3.95/\text{hour} \\
\text{Total Hourly Cost (2 Replicas)} &= 2 \times 3.95 = \text{INR } 7.90/\text{hour} \\
\text{Requests Served per Hour} &= 5.20 \times 3,600 = 18,720 \text{ requests/hour} \\
\text{Cost per 1,000 Requests} &= \left(\frac{7.90}{18,720}\right) \times 1,000 = \text{INR } \mathbf{0.422} \text{ (INR 0.42 per 1,000 requests)}
\end{aligned}$$

7.3 Service Level Objective (SLO) and Error Budget

- **SLO Target:** 99.0% of requests must return HTTP 200 within 30 seconds.
- **Measurement Window:** One 2-minute load test execution (624 requests).
- **Total Error Budget:** $624 \times (1 - 0.99) = 6.24 \approx \mathbf{6}$ requests.
- **Error Budget Consumed:** 0 failed requests (0.0% of error budget consumed; 100% margin retained).

Component	Measured Evidence	Limitation Identified	Proposed Improvement
Data & Storage	92.3% valid kept (2,769/3000)	CPU labelling required hours; local SeaweedFS single pod	Persistent distributed volume with minio multi-tenant replication.
Model Quality	56.0% full frame (vs 88% teacher)	70M model struggles on compound slot boundaries	Distill into Pythia-160M or LoRA adapter with teacher confidence weighting.
Latency & Throughput	p50: 340ms, p95: 720ms, 5.2 rps	CPU PyTorch inference causes head-of-line queuing	Export Pythia model to ONNX Runtime with INT8 quantization or vLLM.
Resource Utilisation	Peak mem: 438 MiB (< 1 GiB limit)	Idle replicas continuously incur compute billing	Enable KEDA (Kubernetes Event-driven Autoscaling) to scale to 0.
Reliability & Recovery	Recovery time: 4.1s, 0.16% error	k3d single-node cluster lacks physical hardware failover	Multi-AZ managed Kubernetes (GKE/EKS) with node affinity rules.
Security & Cost	INR 0.422 per 1k reqs; 100% NACL	Stateless NACL requires broad ephemeral return rules	Deploy AWS Security Groups or Cilium eBPF stateful NetworkPolicies.

Table 4: Comprehensive review of system limitations and proposed engineering enhancements.

7.4 Comprehensive Architectural Review

8 Conclusion

This project successfully validated sequence-level knowledge distillation and containerized orchestration on a local cloud. Fine-tuning a 70M student model yielded 98% JSON validity and 56% full-frame parsing accuracy while eliminating cloud GPU costs. The decoupled microservice architecture achieved sub-second p95 latency under strict resource quotas, validating resilient cloud engineering practices.